

Why regenerate SignWriting OneD?

A SuttonSignWritingOneD.ttf already exists — Slevinski's official build at [sutton-signwriting/font-ttf](https://github.com/sutton-signwriting/font-ttf). Two things make it worth rebuilding:

1. Cubic → quadratic conversion loss

The source SVG glyphs in [sutton-signwriting/font-db](https://github.com/sutton-signwriting/font-db) use cubic Bezier curves. TrueType only supports quadratic Beziers, so FontForge approximates each cubic with a sequence of quadratics during the .ttf export. The approximation has a tolerance of "about one emunit" (`splineorder2.c`) — visible in our earlier round of analysis as a ~1-pixel stroke wobble for any glyph compared between the cubic source and the TTF render.

2. Duplicated outlines for rotations & reflections

Every SignWriting symbol has 16 orientation variants (the last hex digit of the symbol key). In the upstream TTF every variant carries its own full outline. For the cardinal orientations (rot 0, 2, 4, 6, 8, A, C, E) the outline really is just a rotation/reflection of rot 0, and likewise the diagonals (1, 3, 5, 7, 9, B, D, F) are transforms of rot 1 — so 14 of every 16 glyphs can become TrueType composite glyphs that reference one base outline plus a 2×2 transform. Cuts the file by ~67 %.

Following pages walk through what the regeneration produces: the file size and codepoint coverage, the rotation encoding, side-by-side renders of the optimised symbols, and a list of glyphs that still don't quite match upstream so a human can decide whether to accept them.

SignWriting OneD — regeneration report

Comparison of the upstream Sutton SignWriting OneD font vs the font regenerated from font-db's cubic-Bezier SVG sources, with and without our ellipse-replacement optimization.

File size

variant	size		Δ
original (upstream)	8,206,192 bytes	8013.9 KB	
new (no optimisation)	7,537,524 bytes	7360.9 KB	-8.1% vs original
new (ellipse + rotation dedup)	4,920,264 bytes	4804.9 KB	-40.0% vs original

Glyph count

variant	glyphs
original (upstream OneD)	38,316 mapped codepoints
new (no optimisations)	38,316 mapped codepoints
new (ellipse + rotation dedup)	38,316 mapped codepoints

Rotation + reflection encoding (hand symbols)

For each hand $\text{S}_{\text{base}}^{\text{fill}}^{\text{rot}}$, the last hex digit picks one of 16 orientations: 8 rotations of rot 0 plus their mirrors at the rot-8 offset. Diagonals (odd indices) form a parallel sub-family centred on rot 1. Non-hand symbols are not covered by this table — they keep their own outlines.

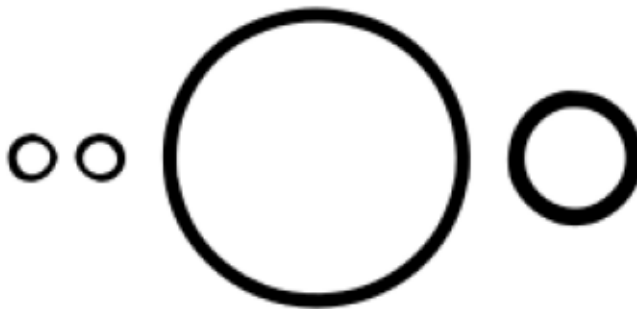
last hex	semantic orientation	composite source	transform applied
0	rot 0°	base (outline kept)	—
1	rot 45°	diag base (outline kept)	—
2	rot 90°	← rot 0	rotate 90°
3	rot 135°	← rot 1	rotate 90°
4	rot 180°	← rot 0	rotate 180°
5	rot 225°	← rot 1	rotate 180°
6	rot 270°	← rot 0	rotate 270°
7	rot 315°	← rot 1	rotate 270°
8	mirror + rot 0°	← rot 0	mirror
9	mirror + rot 45°	← rot 1	mirror
A	mirror + rot 90°	← rot 0	mirror + rot 270°
B	mirror + rot 135°	← rot 1	mirror + rot 270°
C	mirror + rot 180°	← rot 0	mirror + rot 180°
D	mirror + rot 225°	← rot 1	mirror + rot 180°
E	mirror + rot 270°	← rot 0	mirror + rot 90°
F	mirror + rot 315°	← rot 1	mirror + rot 90°

Mirror reverses CCW→CW, which is why rot A's matrix is $\text{mirror} + \text{rot } 270^\circ$ even though the semantic is $\text{mirror} + \text{rot } 90^\circ$.

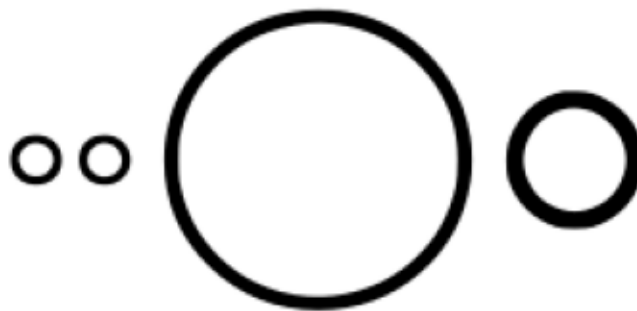
Ellipse optimization — circular glyphs

hand-traced cubic circles replaced by 4-segment Bezier ellipses

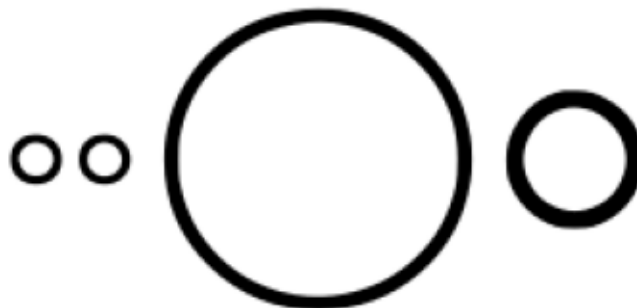
original (upstream OneD)



new (no optimisations)



new (ellipse + rotation dedup)



Rotation dedup — all 16 orientations of S1000{0..f}

even indices = composites of rot 0; odd = composites of rot 1

original (upstream OneD)



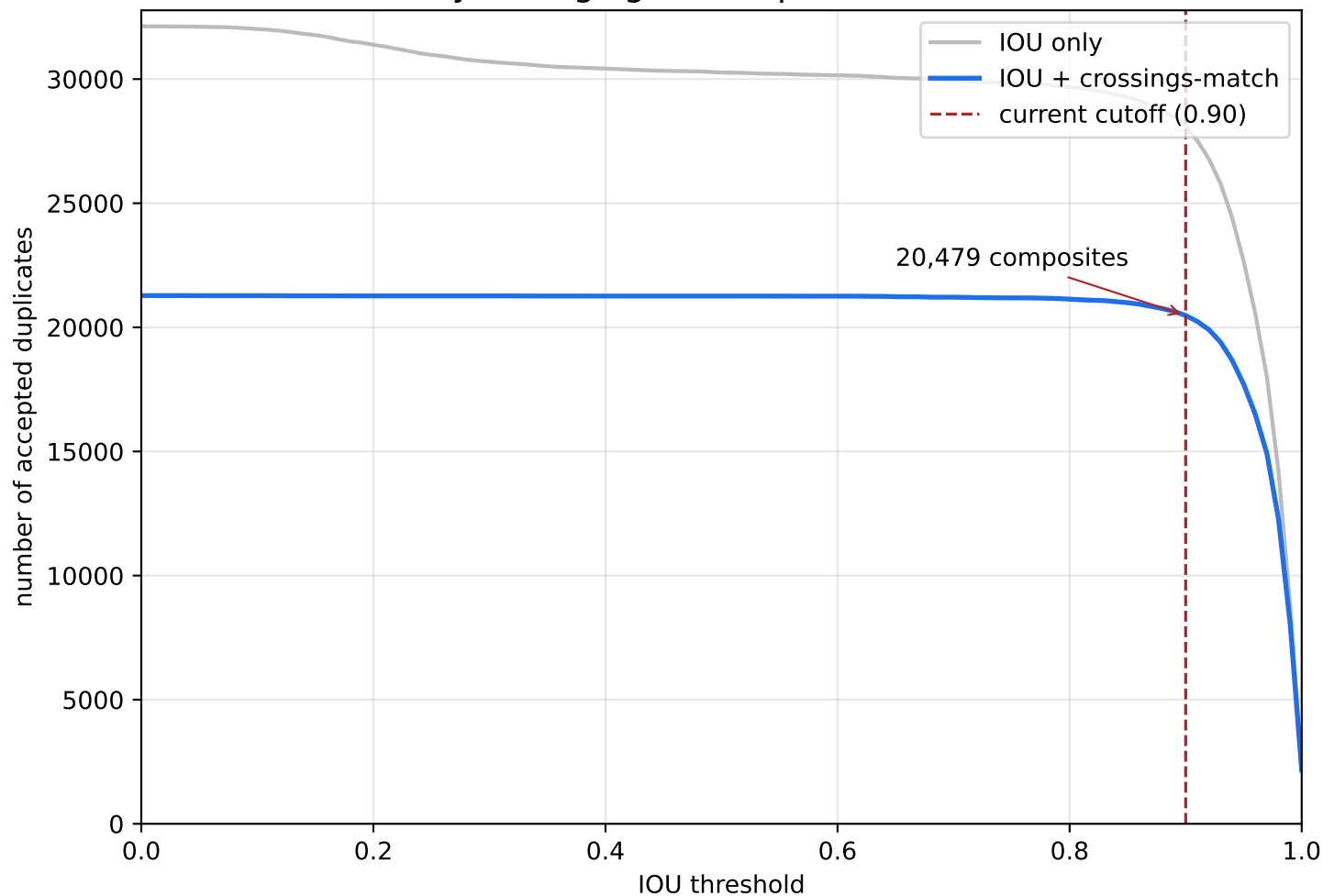
new (no optimisations)



new (ellipse + rotation dedup)



How many siblings get deduped at each threshold



,129. The flat region 0.7-0.9 is the sweet spot: most entries cluster at IOU > 0.9, so raising the cutoff from 0.7 → 0.9 only drops a few th

Section 1 — 10 most certain rejects (lowest IOU among rejects). These stay as outlines.

Section 2 — 10 weakest hand-symbol accepts ($\text{IOU} \geq 0.90$, base < S205). Hands dedup well by D4 transforms.

Section 3 — 10 weakest non-hand accepts. Non-hand families often have hand-redrawn siblings, so borderline accepts here are riskier.

Threshold tuning — the 0.90 cut

Most-certain rejects ($\text{IOU} < 0.90$):

S35e21 → S35e25 via I IOU=0.00



S37600 → S37604 via I IOU=0.00



S35e21 → S35e27 via R180 IOU=0.00



S37600 → S3760e via R180 IOU=0.01



S37601 → S3760d via I IOU=0.02



S35e20 → S35e26 via I IOU=0.00



S37600 → S3760c via I IOU=0.00



S37600 → S37606 via R90 IOU=0.01



S37601 → S37605 via I IOU=0.02

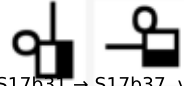


S37600 → S37608 via I IOU=0.03

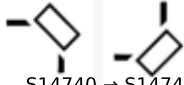


Weakest hand-symbol accepts ($\text{IOU} \geq 0.90$, base < S205):

S13c10 → S13c1e via MR90 IOU=0.90



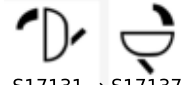
S17b31 → S17b37 via R270 IOU=0.90



S14740 → S14744 via R180 IOU=0.90



S16f30 → S16f36 via R270 IOU=0.90



S17131 → S17137 via R270 IOU=0.90



S14740 → S1474c via MR180 IOU=0.90



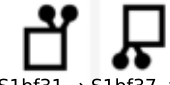
S10310 → S10316 via R270 IOU=0.90



S14a51 → S14a53 via R90 IOU=0.90



S11200 → S11204 via R180 IOU=0.90

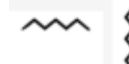


S1bf31 → S1bf37 via R270 IOU=0.90

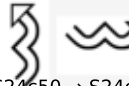


Weakest non-hand accepts ($\text{IOU} \geq 0.90$, base $\geq$ S205):

S22230 → S22232 via R90 IOU=0.90



S29310 → S29316 via R270 IOU=0.90



S24c50 → S24c56 via R270 IOU=0.90



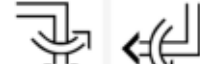
S29d31 → S29d35 via R180 IOU=0.90



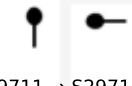
S2dc00 → S2dc04 via R180 IOU=0.90



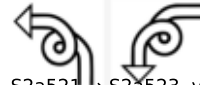
S23e20 → S23e26 via R270 IOU=0.90



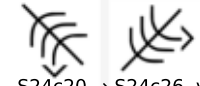
S37150 → S37152 via R90 IOU=0.90



S29711 → S29713 via R90 IOU=0.90



S2a521 → S2a523 via R90 IOU=0.90



S24c20 → S24c26 via R270 IOU=0.90

